

BIOS Guide

The following routines are available in the BIOS:

Name	Address	Description	Inputs	Outputs
os_get_in	0x1000	Get a word from IN_1 or IN_2	R0: dev id	R0: the word
os_disp_out	0x1030	Display a word on OUT_1 or OUT_2	R0: dev id R1: the word	-
os_read_char	0x1050	Get a character from the keyboard	-	R0: the char
os_print_char	0x1080	Show a character on the display	R0: the char	-
os_read_str	0x10a0	Get a string from the keyboard	R0: str addr	-
os_print_str	0x10e0	Show a string on the display	R0: str addr	-
os_mem_set	0x1110	Set a memory location	R0: addr R1: val	-
os_mem_setn	0x1130	Set a memory region	R0: start addr R1: byte count R2: val	-
os_mem_read	0x1150	Read a memory location	R0: addr	R0: val
os_read_int	0x1170	Get a signed integer from the keyboard	-	R0: the integer
os_print_int	0x11d0	Show a signed integer on the display	R0: the integer	-
os_read_uint	0x1230	Get an unsigned integer from the keyboard	-	R0: the integer
os_print_uint	0x1280	Show an unsigned integer on the display	R0: the integer	-
os_read_float	0x1330	Get a floating point number from the keyboard	-	R0: the 'float'
os_print_float	0x16b0	Show a floating point number on the display	R0: the 'float'	-
os_exit	0x1a90	Terminate program & display non-zero error codes	R0: error code	-
os_play_tune	0x1ac0	Play a tune on the sound card	R0: notes addr R1: pauses addr	-
os_delay	0x1af0	Delay for 'duration'	R0: duration	-

Any programs using these routines should include the following definitions:

```
.equ os_get_in      0x1000
.equ os_disp_out    0x1030
.equ os_read_char   0x1050
.equ os_print_char  0x1080
.equ os_read_str     0x10a0
.equ os_print_str    0x10e0
.equ os_mem_set      0x1110
.equ os_mem_setn     0x1130
.equ os_mem_read     0x1150
.equ os_read_int     0x1170
.equ os_print_int    0x11d0
.equ os_read_uint    0x1230
.equ os_print_uint   0x1280
.equ os_read_float   0x1330
.equ os_print_float  0x16b0
.equ os_exit         0x1a90
.equ os_play_tune    0x1ac0
.equ os_delay        0x1af0
```

The 'IN_x' and 'OUT_x' Device ID's are:

```
.equ IN_1  3
.equ IN_2  7
.equ OUT_1 6
.equ OUT_2 10
```

When using these routines in 'ASM' code the 'Input Registers' should be set up, then a 'call func' instruction used, with any result then being available in R0 following the 'call'.

And when using 'RMG' code the 'Input Registers' are used as arguments to the function call, with any result being assigned as required.

Here are 'ASM' and 'RMG' examples of 'os_get_in' and 'os_disp_out' being used:

```
; Transfer IN_1 -> OUT_2
ldi r0,IN_1
call os_get_in
mov r0,r1
ldi r0,OUT_2
call os_disp_out
end
```

```
// Transfer IN_1 -> OUT_2
val = os_get_in(BIOS_IN1);
os_disp_out(BIOS_OUT2, val);
```